如何绘制路径

> 路径的使用基本就看对画布路径相关API的熟悉程度，但无论熟悉与否，知道它们的存在是最基本的要求。

1）认识基本API

```
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');

canvas.width = 500;
canvas.height = 500;

ctx.beginPath();
ctx.moveTo(100, 100);
ctx.lineTo(300, 100);
ctx.lineTo(300, 300);
ctx.lineTo(100, 300);
ctx.closePath();

// 路径填充，默认#000000
ctx.fill();
```

`beginPath()`

开始一段新的路径。

> 源码中通过 `path.clear()` 清除原有路径数据来开启一段新的路径，所以如果在开启新路径时忘记调用该API，旧的路径点会参与下次绘制，切记。

`closePath()`

闭合当前路径，如果当前路径未闭合，则从当前路径点连接到路径起始点以形成闭合路径。

> 对于填充 `fill` 而言它必须是一个闭合区域，所以即使你不调用 `closePath` ，还是会连接起始点以填充闭合区域，对于描边 `stroke` 则不影响。
>
> 如果只看名字以为是关闭当前路径的意思，实际上调用后当前路径数组并未清空，请勿错把“闭合”看成“关闭”。

`moveTo(x, y)`

要移动到 `(x,y)` 开始绘制。跟写字一样，每写一个新的笔画都要移动到新的起笔处。

`lineTo(x, y)`

从当前路径点画一条线到 `(x,y)` 。

> 注意：如果之前调用前没调用过 `moveTo` ，也就是没确定当前路径点，则调用后效果等同于 `moveTo` ，而不是拿 `(0,0)` 当起始点。

2）形状路径API

https://codepen.io/huanjinliu/pen/VwVxgdq?editors=1010

`rect(x, y, width, height)`

矩形。

`roundRect(x, y, width, height, radius)`

圆角矩形， `radius` 的设置与 `css border-radius` 设置规则相似但并不完全一样：

```
// all-corners
roundRect(0, 0, 100, 100, 8);
// all-corners
roundRect(0, 0, 100, 100, [8]);
// [top-left & bottom-right, top-right & bottom-left]
roundRect(0, 0, 100, 100, [4, 8]);
// [top-left, top-right & bottom-left, bottom-right] !与css border-radius不同
roundRect(0, 0, 100, 100, [4, 4, 8]);
// [top-left, top-right, bottom-right, bottom-left]
roundRect(0, 0, 100, 100, [4, 4, 8, 8]);
```

`ellipse(x, y, radiusX, radiusY, rotation, startAngle, endAngle[, counterclockwise])`

椭圆形。

`arc(x, y, radius, startAngle, endAngle[, counterclockwise])`

圆弧，可实现圆形。

`arcTo(x1, y1, x2, y2, radius)`

圆弧，无法实现圆形。

`quadraticCurveTo(cpx, cpy, x, y)`

二次贝塞尔曲线。

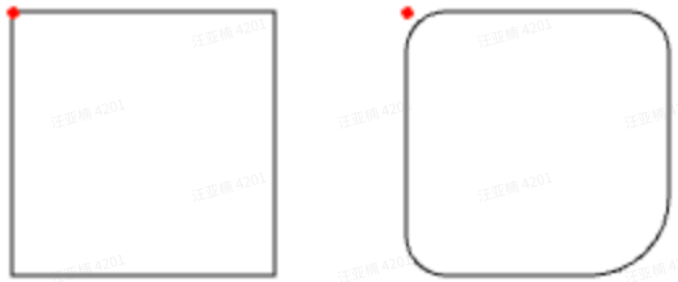
`bezierCurveTo(cp1x, cp1y, cp2x, cp2y, x, y)`

三次贝塞尔曲线。

拓展认识

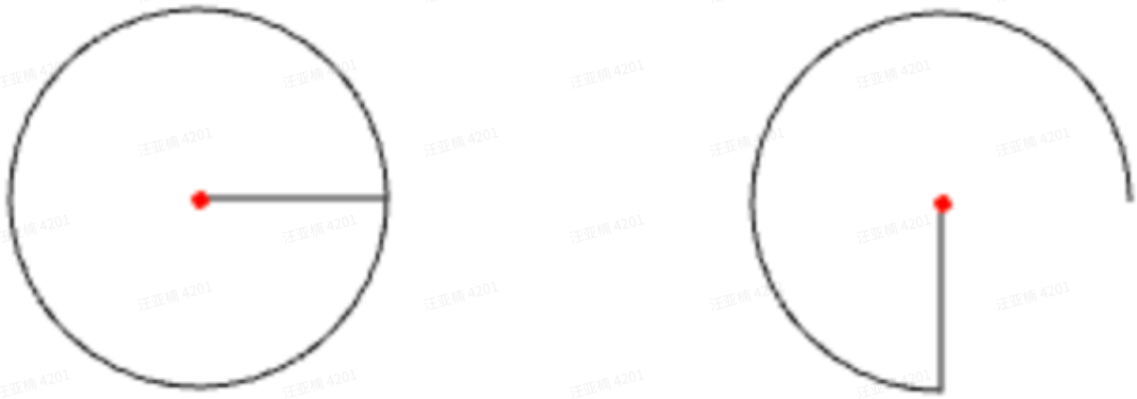
形状路径API更多细节用到时可直接参考MDN。但是以下一些文档未提及的可以先了解下：

1. 矩形、圆角矩形会自动将路径起始点移动到形状左上角。相当于绘制前，自动调用 `moveTo(左上角坐标)` ，所以没必要再手动调用 `moveTo` 了。



1. 椭圆形、圆弧（arc）的绘制圆中心点是路径起始点的位置，所以当你画完圆后给圆描边后：

```
// 圆弧（可用于生成圆形）
ctx.moveTo(100, 250);
// arc(x, y, radius, startAngle, endAngle)
ctx.arc(100, 250, 50, 0, 2 * Math.PI);
ctx.stroke();
```

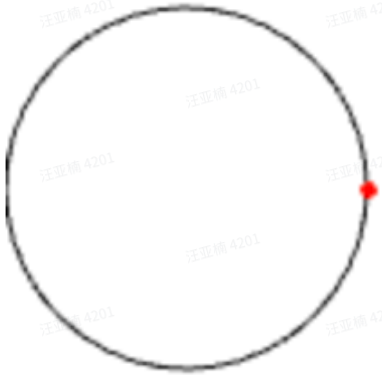


你会惊奇发现这划线是从中心出来的，至于它连接了圆的哪个位置取决于它的 `startAngle` ，如上图。

如果你不想要中间的线出现，请对圆的绘制起点做适当偏移，如上第一张图你就应该向左偏移一个圆半径的位置：

```
ctx.moveTo(100, 250);
// 向左偏移一个圆半径的位置
ctx.arc(100 - 50, 250, 50, 0, 2 * Math.PI);
```

这时起点会与圆的绘制起点相重合。



1. `roundRect` 和 `ellipse` 仍未支持IE，但其他主流浏览器皆良好支持，所以如果要考虑IE，还是自行实现吧。
2. 不通过 `beginPath` 清除路径，连续调用 `rect` API会出现一种情况：画布不绘制了，原因是：

在HTML5 Canvas中，每次调用 `rect` 方法都会在内存中创建一个新的路径对象。这个路径对象包含了绘制矩形所需的所有信息，如位置、大小等。如果你在一个循环中连续调用rect()方法，而没有清除或重置路径，那么每次调用都会创建一个新的路径对象，这些对象会累积在内存中。而内存是有限的，当创建的路径对象过多，占用的内存超过了可用内存时，就会发生内存溢出。这时，浏览器可能会选择停止绘制，以防止程序崩溃，这就是为什么绘制会变得无效。

3）认识Path2D

在上面的路径使用中，我们会发现 `context2D` 对路径的绘制是一次性的，如果我们要绘制一样的路径，仍需要执行一样的路径创建流程，如果这个路径中间生成需要非常复杂的运算，那多次重复的生成无异于在浪费计算机资源也会导致性能下降。

那要如何做呢？**缓存路径**。

`Path2D` 对象的主要作用就是对路径做缓存，以达到多次复用路径的作用。

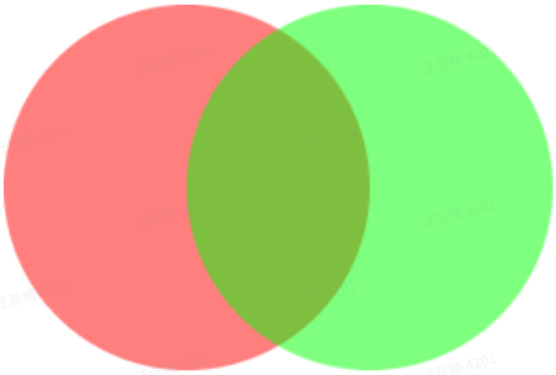
```
// 通过Path2D记录原点为左上角的圆的路径
const circle = new Path2D();

circle.arc(100, 100, 100, 0, 2 * Math.PI);

// 使用路径绘制红色圆
ctx.fillStyle='rgba(255, 0, 0, 0.5)';
ctx.fill(circle);

```

```
// 使用路径绘制绿色圆
ctx.translate(100, 0)
ctx.fillStyle='rgba(0, 255, 0, 0.5)';
ctx.fill(circle);
```



`Path2D` 对象包含context2D绘制中所有路径绘制相关的API，这里大致可以把 `Path2D` 看成是一个代理对象：

```
class Path2D {
  cachePath;
  [API]() {
    // 执行context2D的api并缓存最新路径
    this.cachePath = context2D.[API]();
  }
}
```

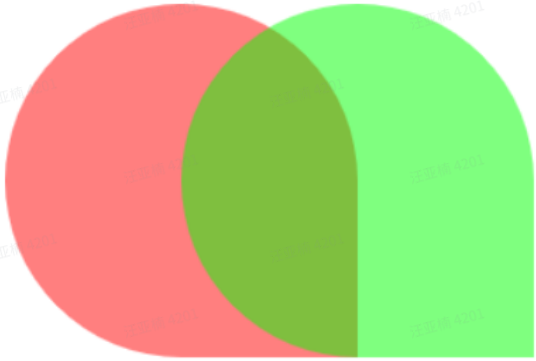
`Path2D.addPath(path [, transform])`

`path2D` 对象的专有API，用于添加路径进当前路径，从而组合成新的路径。

```
const path = new Path2D();
path.arc(100, 100, 100, 0, 2 * Math.PI);

const square = new Path2D();
square.rect(100, 100, 100, 100);

// 向路径里面再添加一个矩形路径，参数二是变换矩阵参数
path.addPath(square, new DOMMatrix([1, 0, 0, 1, 0, 0]));
```



4）用Path2D加载svg矢量路径。

`context2D` 中是没有直接加载svg文件API的，只能通过 `drawImage` 当做图像加载，但这个时候只能作为图像并不能为矢量路径做直接操作，比如改路径颜色。

但 `Path2D` 提供了一种方式让我们直接使用svg中的矢量路径，如下：

```
const path = new Path2D("M10 10 h 80 v 80 h -80 Z");
```

参数是SVG标签 `path` 中的路径：d。

如果svg中含有基础形状，则需要先转成路径。